DIGITAL TWIN ASSISTED DYNAMIC NETWORK SLICING USING SSL, TGNN AND MARL FOR AI-NATIVE 6G NETWORKS

A Thesis

Submitted in partial fulfilment for the Degree of

BACHELOR OF TECHNOLOGY

in

COMPUTER SCIENCE AND ENGINEERING (ARTIFICIAL INTELLIGENCE)

by

Keerthivasan S V (CB.SC.U4AIE23037)
Krish S (CB.SC.U4AIE23040)
Sriranjana C (CB.SC.U4AIE23066)
Thrishala S N (CB.SC.U4AIE23072)



DEPARTMENT OF ARTIFICIAL INTELLIGENCE

AMRITA VISHWA VIDYAPEETHAM

COIMBATORE CAMPUS (INDIA)

October 2026

DEPARTMENT OF ARTIFICIAL INTELLIGENCE

AMRITA VISHWA VIDYAPEETHAM

COIMBATORE CAMPUS



BONAFIDE CERTIFICATE

This is to certify that the thesis entitled "**Digital Twin Assisted Dynamic Network Slicing using SSL, TGNN and MARL for AI-Native 6G Networks**" submitted by **Keerthivasan S V (CB.SC.U4AIE23037)**, **Krish S (CB.SC.U4AIE23040)**, **Sriranjana C (CB.SC.U4AIE23066)**, and **Thrishala S N (CB.SC.U4AIE23072)** for the award of the **Degree of Bachelor of Technology** under the **Faculty of Engineering** is a bonafide record of the work carried out by them under my guidance and supervision in the Department of Artificial Intelligence, Amrita Vishwa Vidyapeetham, Coimbatore Campus.

Submitted for the examination held on _____.

INTERNAL EXAMINER **EXTERNAL EXAMINER**

Dr. Sundaresan S

Thesis Advisor

Department of Artificial Intelligence,

Amrita Vishwa Vidyapeetham, Coimbatore Campus, India.

DEPARTMENT OF ARTIFICIAL INTELLIGENCE
AMRITA VISHWA VIDYAPEETHAM
COIMBATORE CAMPUS

DECLARATION

We, **Keerthivasan S V., Roll. No: CB.SC.U4AIE23037**, **Krish S., Roll. No: CB.SC.U4AIE23040**, **Sriranjana C., Roll. No: CB.SC.U4AIE23066** and **Thrishala S N., Roll. No: CB.SC.U4AIE23072** hereby declare that this thesis entitled "**DIGITAL TWIN ASSISTED DYNAMIC NETWORK SLICING USING SSL, TGNN AND MARL FOR AI-NATIVE 6G NETWORKS**" is the record of the original work done by us under the guidance of **Dr. Sundaresan S**, Assistant Professor, Department of Artificial Intelligence, Amrita Vishwa Vidyapeetham, Coimbatore Campus. To the best of our knowledge, this work has not formed the basis for the award of any degree/diploma/associateship/fellowship or a similar award to any candidate in any University.

Place: **Signature of the Students**
Date:

COUNTERSIGNED

Dr. K.P Soman
Professor and Dean,
Department of Artificial Intelligence
Amrita Vishwa Vidyapeetham,
Coimbatore Campus,
India.

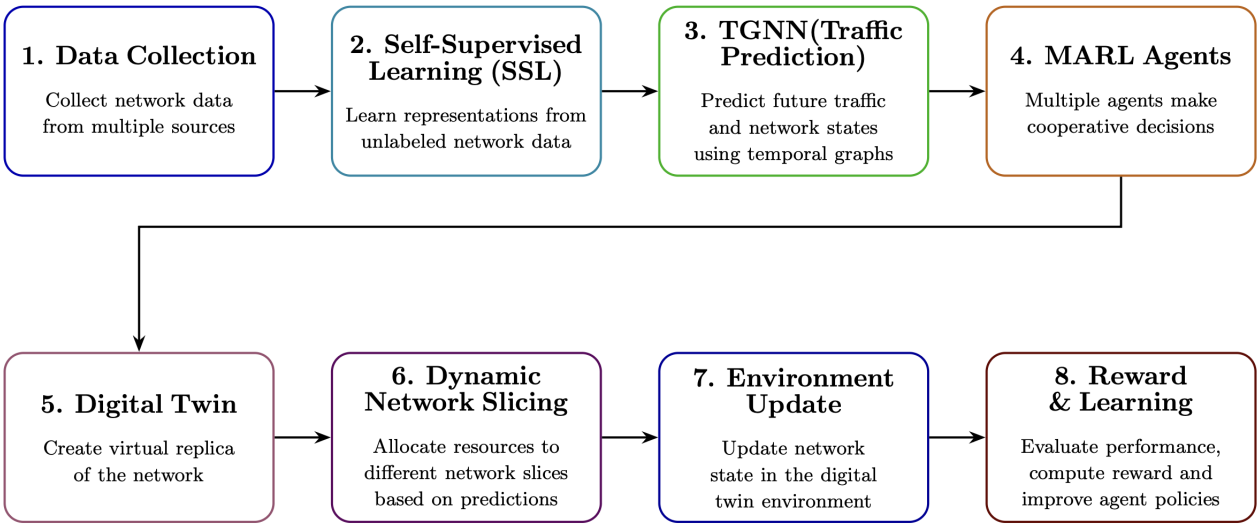


Figure 4.1: Architecture of the proposed system

4.3 Problem Formulation

Each cell is controlled by one agent. The agent splits the capacity of its cell into four portions for each of eMBB, URLLC, mMTC and idle and applies the selected portions to the next interval, and so on. Each agent in MARL takes an action only based on 17 local features for their own cell. The load of neighbouring cells diminishes the capacity of a cell (Section 4.4), which means that the decisions of the agents are coupled, and the task is a decentralised multi-agent control problem.

The reward of the agent of cell i in interval t is

$$R_t^i = 0.7\, S_t^i + 0.3\, F_t^i - 0.1\, H_t^i, \tag{4.1}$$

where S_t^i is a weighted SLA score, F_t^i is a fraction of demand that is satisfied and H_t^i is a reconfiguration churn. The weights are design choices. The reward therefore favours meeting the SLA targets of the three slices first, then serving demand, and penalises changes of the allocation. During training, each agent's reward is mixed with those of its neighbours (the team reward).

4.4 The Unified Slicing Environment

The environment brings together real traffic on a real spatial layout, per-slice demand, an allocation action, and the resulting QoS, SLA outcome and reward for each cell and each 10-minute interval. It is implemented in the `src/unified` folder of the project repository. Table 4.1 shows which components are real, which are calibrated and which are modelling assumptions.

The traffic and the grid layout are real. The mapping from activity type to slice and the scaling of each slice to the 100 Mbps cell capacity are assumptions, because the Milan data

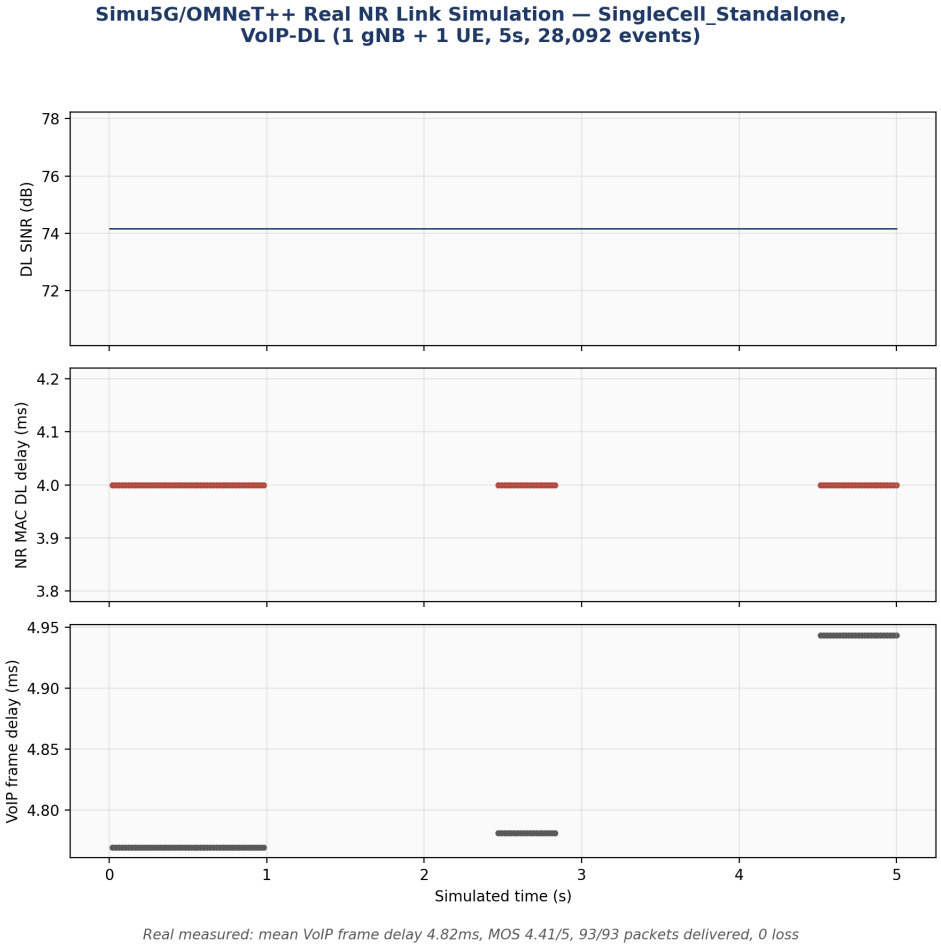


Figure 4.2: Reference Simu5G NR run (one gNodeB, one UE, 5 s, downlink VoIP)

4.8 Summary

The unified environment has real traffic and topology from Milan, slice demand, a latency and jitter model calibrated on 6G RAN telemetry, interference from neighbouring cells, and the reward of Equation (4.1). It allows forecasting and allocation to be tested together on the same traffic and compared with equal-split, proportional and expert policies, including tuned versions. Its main limit is the surrogate QoS model, which Chapter 8 compares with a packet-level simulator.

Table 5.1: Macro-F1 on 81,731 unseen 5G-NIDD flows, by number of labelled flows (mean of five label draws up to 5,000 labels, two above)

Labels	LGBM raw	MLP raw	Logit raw	Masked emb.	Masked fine-tuned	SCARF emb.	SCARF fine-tuned	Raw + SCARF
50	0.643	0.694	0.704	0.627	0.684	0.594	0.688	0.690
200	0.852	0.858	0.853	0.763	0.834	0.713	0.850	0.848
1,000	0.905	0.895	0.874	0.859	0.881	0.842	0.894	0.888
5,000	0.919	0.911	0.909	0.901	0.898	0.880	0.901	0.915
20,000	0.932	0.924	0.923	0.913	0.923	0.907	0.925	0.924
all	0.936	0.925	0.926	0.912	0.927	0.910	0.929	0.926

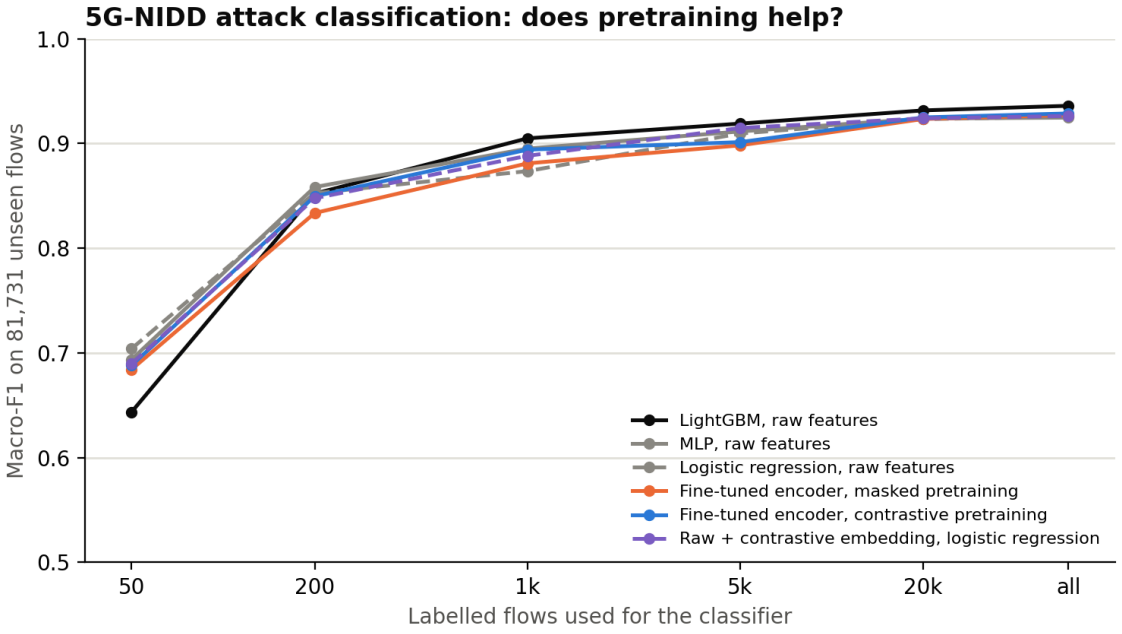


Figure 5.1: Macro-F1 against the number of labelled flows for raw features and pretrained encoders

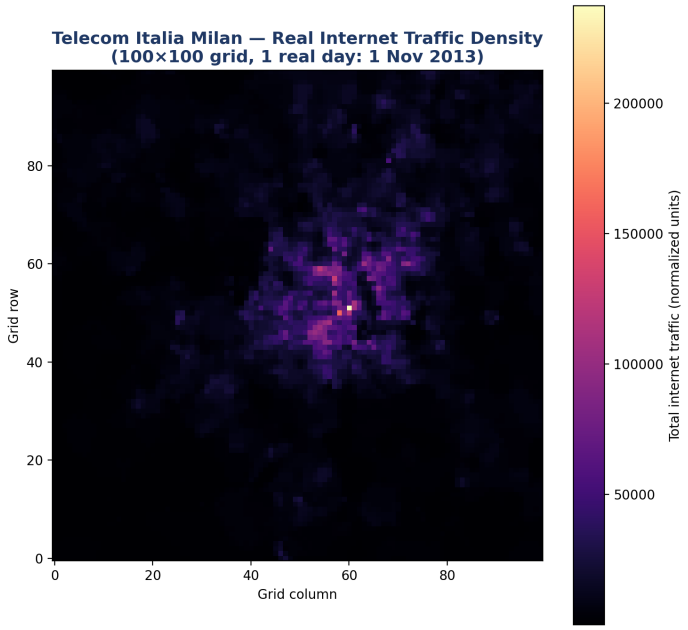


Figure 6.1: Real traffic on the Milan grid

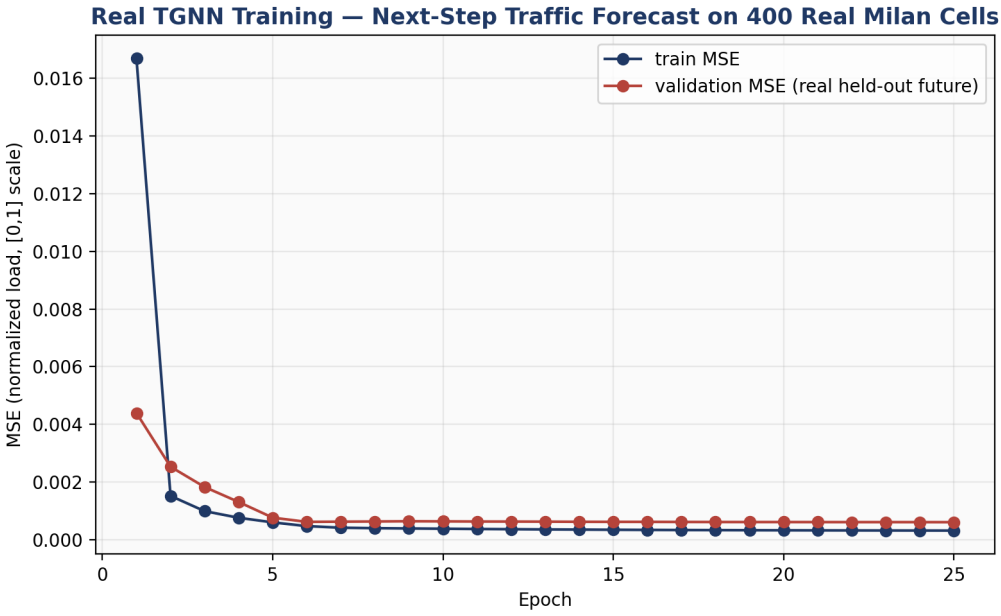


Figure 6.2: Training and validation error of the first Milan forecaster

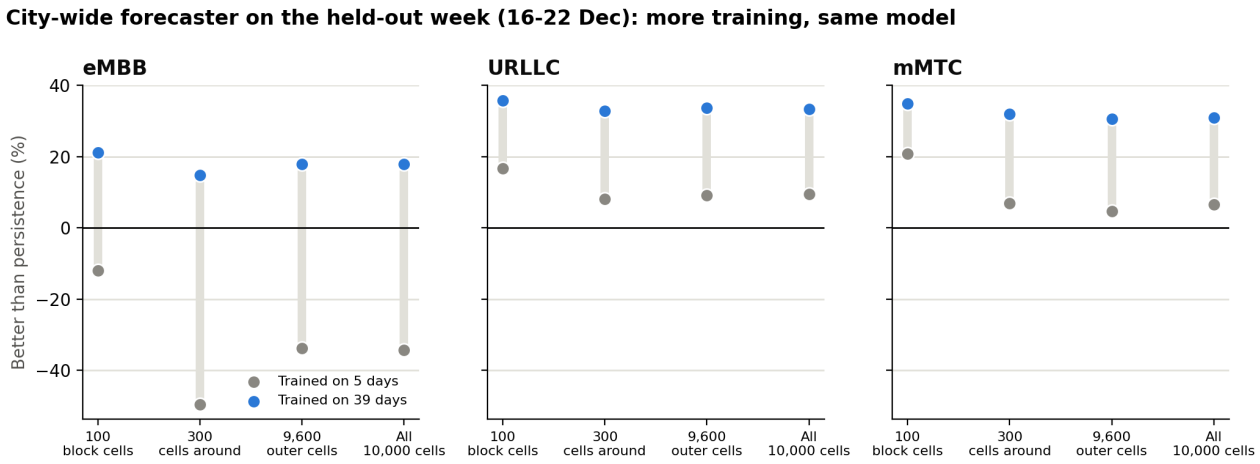


Figure 6.3: City-wide forecaster on the held-out week: five days of training against 39 days

Table 6.4: Mean squared error of simple baselines and the TGNN on all 10,000 cells ($Mbps^2$), held-out week

Forecaster	**eMBB**	**URLLC**	**mMTC**
Repeat the last value (persistence)	1.449	0.127	0.061
Same time on the previous day	11.799	0.666	0.193
Average for that time of day	19.736	0.522	0.152
TGNN, trained on 39 days	1.190	0.084	0.042

6.7 Limitations of the Forecasting Study

Each forecaster was trained once, so there is no estimate of seed-to-seed variation. The study also does not separate the contributions of per-cell scaling, patch sampling, the extra inputs and the graph aggregation, and it has no graph-free baseline (a GRU on each cell alone) or published spatio-temporal baselines such as STGCN. These ablations would show which part of the model does the work, and are listed as future work in Chapter 10. The value of the forecast to the controllers is measured in Chapter 8.

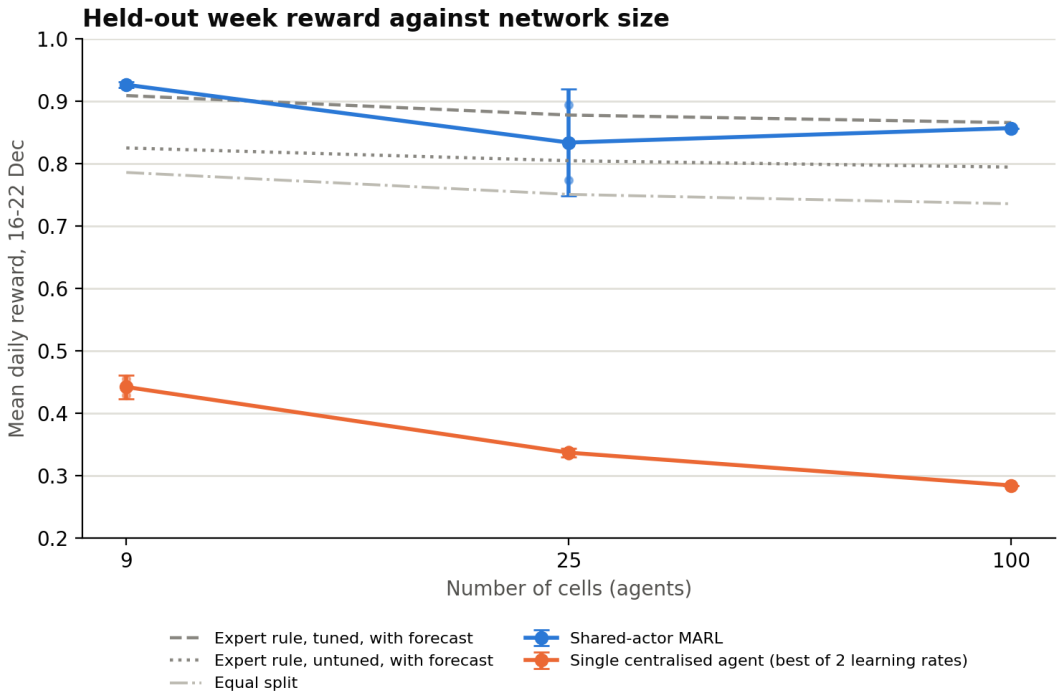


Figure 7.1: Reward against network size: shared MARL, the single agent and the reference rules

and the rest to eMBB. Its default targets were chosen by hand. Two tuned versions were therefore built, using only the training days.

- **Simple tuned rule.** Two utilisation targets (URLLC and mMTC) are chosen by grid search on the training days. At 25 cells the search picks 0.4 and 0.4 and raises the held-out reward of the forecast-fed rule from 0.804 to 0.878.

- **Rich tuned rule.** The targets are chosen separately for each four-hour block of the day and each of three cell classes (by traffic level), 18 pairs in total.

Table 7.1 shows that at 1,000 episodes the shared MARL controller is clearly better than the untuned rules but loses to the simple tuned rule at 25 and 100 cells. The budget was then raised to a long protocol: 4,000 episodes, the checkpoint with the best validation reward, and an average of several seeds' actors in logit space. Table 7.3 reports it.

Against the simple tuned rule, the learned controller is better on every one of the seven test days at 9, 25 and 100 cells, by 0.023 to 0.024 in reward. Against the rich tuned rule the difference is within about 0.005 and the intervals include zero at 25 and 100 cells. The off-policy controller therefore matches a carefully tuned, time-aware rule and beats a simply tuned one; it does not clearly beat the strongest hand-built rule. The on-policy learners of Section 7.7 do go beyond it. The 100-cell result rests on one seed, and its interval describes the seven days only.

Table 7.3: Long protocol (4,000 episodes, checkpoint chosen on validation days, actors averaged in logit space) against the tuned expert rules, held-out week (difference in reward, 95% CI, days better out of 7)

Cells	MARL (actors)	Simple tuned expert	MARL minus simple	Rich tuned expert	MARL minus rich	True-next oracle
9	0.933 (3)	0.909	+0.024 ± 0.004 (7/7)	0.928	+0.005 ± 0.004 (6/7)	0.945
25	0.904 (3)	0.877	+0.026 ± 0.005 (7/7)	0.899	+0.005 ± 0.003 (7/7)	0.920
100	0.888 (1)	0.865	+0.023 ± 0.006 (7/7)	0.887	+0.001 ± 0.003 (3/7)	0.908

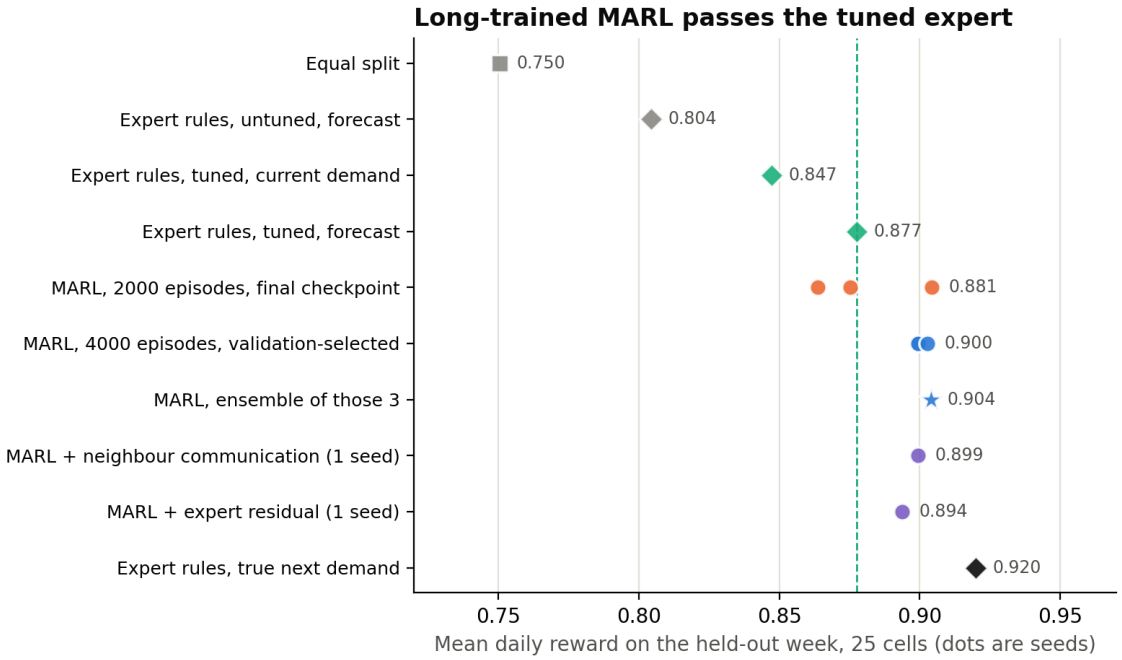


Figure 7.2: Long-protocol MARL against the tuned rules on the held-out week

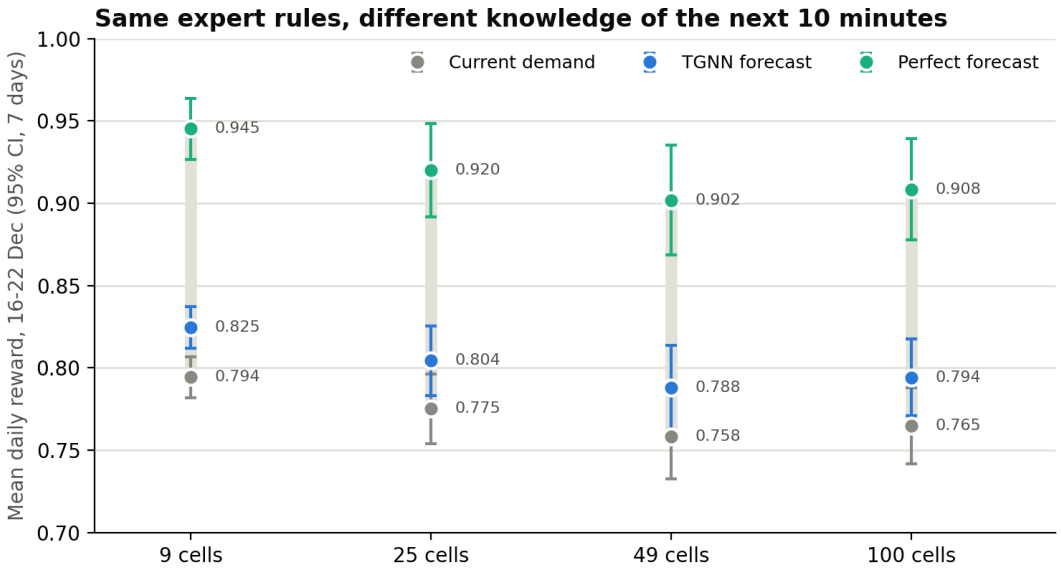


Figure 8.1: Value of prediction for the expert rule at four network sizes

The forecast improves the expert rule by about 0.03 in reward at every network size, on all seven test days, with intervals of about 0.004 to 0.008. It recovers about one fifth of the gap to perfect knowledge. An earlier figure of 0.05 came from two test days and a forecaster trained on five days; with 39 training days and a full week of testing the benefit is smaller and better established. Using the value of the previous day at the same time as the forecast does not work: its mean reward is 0.74 to 0.78 at every size, below that of current demand.

8.3 The Forecast and the Learned Controller

The forecast was also given to the MARL controller as an extra input. Table 8.2 shows that, at this training budget and with three seeds, it made no reliable difference. Giving the controller the true next-interval demand did not help either.

Table 8.2: MARL with and without forecast input (25 cells, three seeds, mean ± sample standard deviation)

Input to the MARL controller	**Held-out reward**
No forecast	0.778 ± 0.048
Forecast from the first (5-day) TGNN	0.765 ± 0.051
Forecast from the retrained TGNN	0.772 ± 0.073
True next-interval demand	0.762 ± 0.057

The three rewards differ by less than their sample standard deviations of 0.05 to 0.06, so the experiment shows neither a clear gain nor a clear loss. Under shifted traffic its effect on the learned policy also changes sign (Section 7.8).We therefore claim that forecasting helps the rule-based controller and that it has not been shown to help the learned one. The forecast is not wasted: the Digital Twin gate of Section 8.7 uses it to predict the outcome of a proposed allocation.

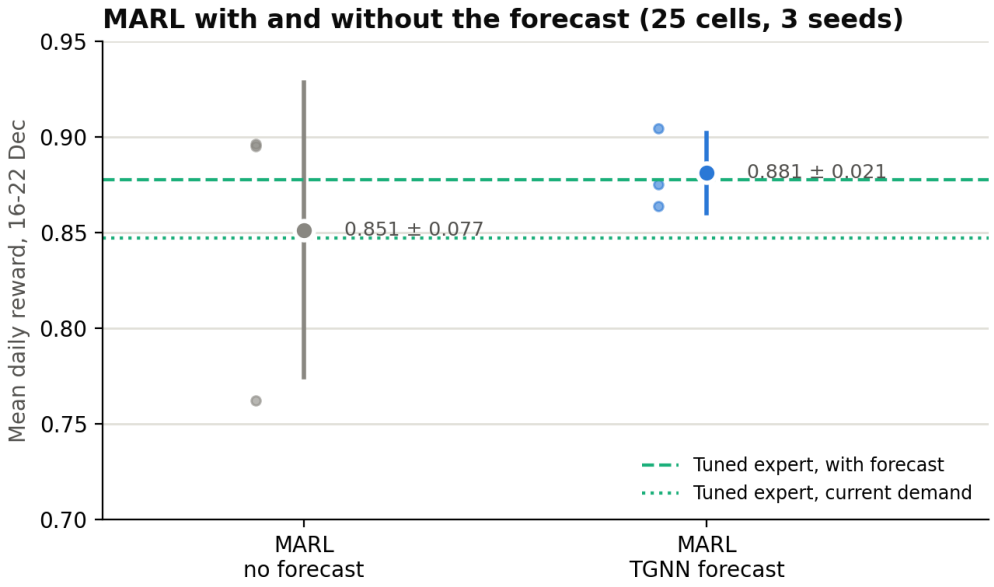


Figure 8.2: Learned controller with and without the forecast

8.4 Controllers Across the Whole City

The three controllers of the Slice Twin monitor (Chapter 9) were run on all 10,000 Milan cells for one day, Wednesday 6 November 2013, with scripted incidents. Static divides capacity equally. Reactive applies the expert rule to the current demand. Predictive applies it to the city-wide forecast. This day was not used to train the forecaster.

Table 8.3: Controllers on all 10,000 cells, Wednesday 6 November 2013 (reward of the whole city, the dense block and the outer city)

Controller	**SLA met (eMBB / URLLC / mMTC)**	**Whole city**	**Dense block**	**Outer city**
Static	0.919 / 0.997 / 1.000	0.974	0.709	0.977
Reactive	0.975 / 0.711 / 0.568	0.839	0.735	0.840
Predictive	0.975 / 0.786 / 0.601	0.868	0.773	0.869

In the dense block the Predictive controller scores highest (0.773 against 0.735 and 0.709), as in Section 8.2. Across the city, however, Static scores highest (0.974 against 0.868 and 0.839). Most cells are lightly loaded, and the rule gives very little capacity to URLLC and mMTC when their traffic is close to zero, so those slices miss their targets when traffic returns. Since the outer city contains most cells, this outweighs the benefit of forecasting. An earlier run reported a city-wide score of 0.935 for the Predictive controller; that result came from over-allocation caused by a bias of the block-trained forecaster in the outer cells, and it was replaced by the table above. Because the comparison covers a single day, it shows the behaviour of the controllers and is not a ranking.

Table 8.9: Simu5G load sweep (10 UEs, one gNodeB, downlink capacity 56.1 Mbps, 1000-byte messages, 6 s, mean of three repetitions)

Utilisation	**Median delay (ms)**	**95th-percentile delay (ms)**	**Received (Mbps)**
0.11	4.5	5.0	6.0
0.21	4.5	4.9	12.0
0.36	4.4	5.0	20.0
0.54	4.5	5.0	30.0
0.68	4.7	6.9	38.0
0.78	4.6	6.8	44.0
0.86	6.0	6.8	48.0
0.93	6.1	434.5	50.5
1.00	5.9	1376.2	50.1
1.07	5.8	780.8	50.7
1.25	11.1	211.9	55.6

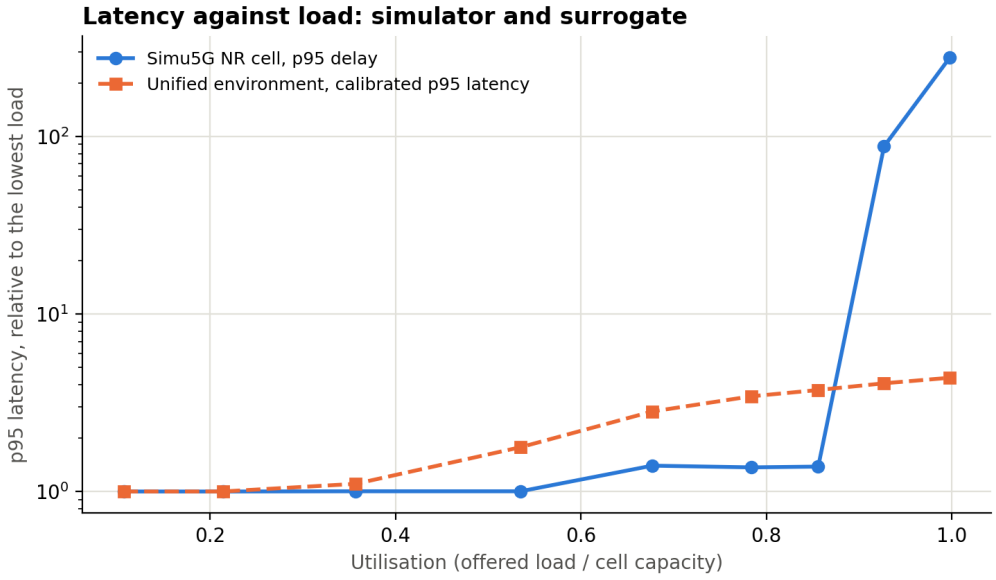


Figure 8.3: Delay against load: Simu5G and the surrogate of the environment (normalised)

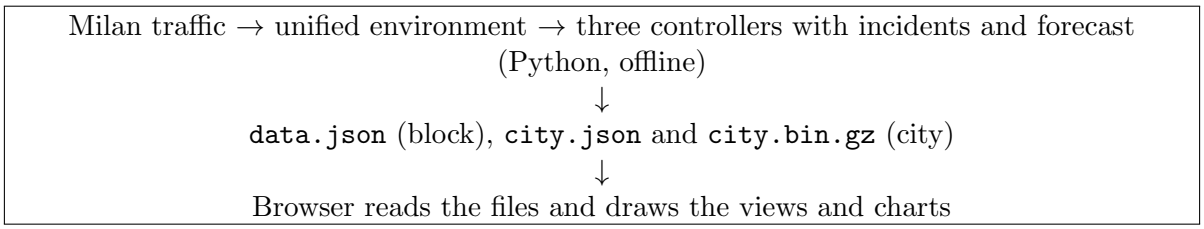


Figure 9.1: Data flow of the Slice Twin monitor

to 2 for each slice meeting its SLA, bit 3 for an event and bit 4 for an outage. The compressed binary file is a few megabytes, which lets the browser handle 10,000 cells over 144 intervals and three controllers.

How to run it. From the repository root, `python -m http.server 8765 --directory app/slice_twin` serves the monitor at `http://localhost:8765`. To regenerate the data, run `showcase.py`, then `forecast_city_v2.py` once to train the city forecaster, then `showcase_city.py`; the Milan files must be on the project drive. A GitHub Actions workflow publishes the monitor, the results dashboard and the project brief to GitHub Pages whenever the site or monitor folders change.

Demonstration. Figure 9.2 shows the city view at 17:30 on the demonstration day. On the left the cells are coloured by real traffic, with the storm-damage incident in the south marked, and on the right by SLA misses under the Reactive controller.

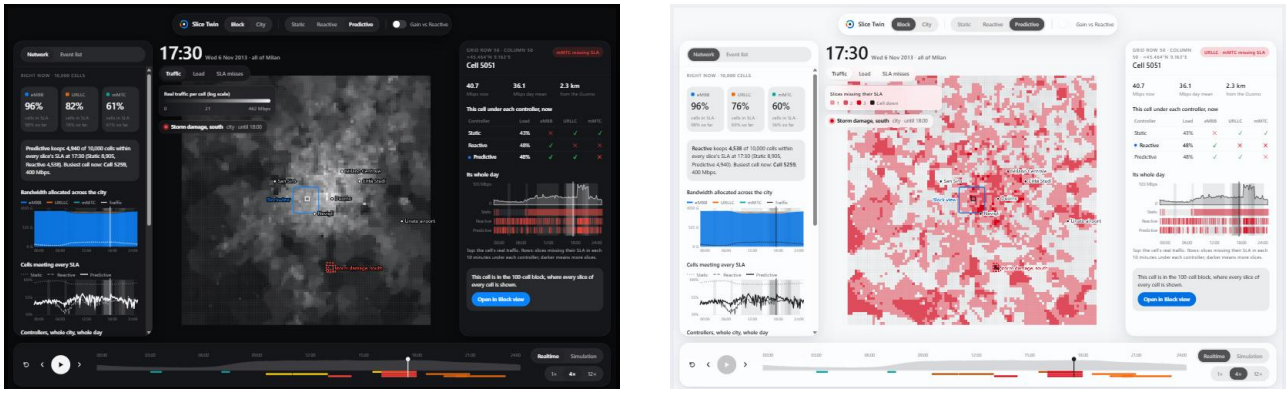


Figure 9.2: City view at 17:30: cells coloured by real traffic (left, dark theme) and by SLA misses under the Reactive controller (right, light theme).

9.5 Contribution to the Project

The Slice Twin monitor is the working product of the project, supported by the code, the unified environment and the dataset in the repository. It also serves as a demonstration tool. A viewer can see that the predictive controller helps in the dense centre, that the static split holds up better on quiet outer cells, and how each incident affects each slice. Its limits are